

MODULE 13 · PART 4 - SEQUENCING AND TIME

Gaps and Islands

Runs of consecutive things. Three canonical solutions, and most people know one.

PART 4 of 8 MODULE 13 of 26 RUNS about 38 min

Why this is the technique to learn

- ◆ **Three of the capstone's ten deliverables are this.** Trading spells, app sessions, and loyalty tier spells.
- ◆ **It has no keyword and no function.** Nothing in SQL is called this, so you cannot look it up by name.
- ◆ **Which is why people reinvent it badly,** usually as a self-join per row, usually at the wrong grain.
- ◆ **And it is built entirely from Part 2.** A LAG, a running SUM, and a GROUP BY. Nothing new is introduced here.

THE TAKEAWAY

The hardest technique in this course is made of three things you already have. What makes it hard is that nothing prompts you to combine them.

There is no such thing as **a run**. There are only rows, and rows do not know about each other.

Rows that belong to one run need **a group number**. Everything here is a way to invent one.

Three questions, one problem

THE QUESTION

How long was this store trading?

How many app sessions did this member have?

When did this member hold Gold status?

Which weeks had no sales at all?

WHAT BREAKS A RUN

A missing day. The store was closed.

Thirty minutes of silence.

The value changed to something else.

The same problem, asking for the gaps instead.

THE TAKEAWAY

Only the second column differs. Everything else about these four queries is identical, which is why learning one of them teaches you all four.

Reading an islands query out loud

WRITTEN	SAID
<code>PARTITION BY store_key</code>	runs of days, within one store
<code>ORDER BY cal_date</code>	in date order
<code>cal_date - LAG(cal_date) OVER w > 1</code>	a gap of more than a day starts a new run
<code>SUM(starts_run) OVER (... UNBOUNDED PRECEDING)</code>	how many runs have started so far
<code>GROUP BY store_key, run_no</code>	collect the rows of each run together

THE TAKEAWAY

Five phrases, and the third one is the business rule. If you cannot say the third one in English, you have not decided what the report means yet.

The shape of every answer

- ◆ **Order the rows within a partition.** By date, by timestamp, by sequence. Module two applies in full.
- ◆ **Invent a group number** that is constant within a run and changes between runs. This is the whole difficulty.
- ◆ **Then GROUP BY it**, taking the minimum and maximum of the ordering column. That is the island.
- ◆ **Three lines at the end, every time.** Only the middle step differs between the three solutions.

THE TAKEAWAY

If you can state what breaks a run in one sentence, you can write the query. Almost every difficulty here is failing to state that sentence first.

The data, for the rest of this module

```
1 store_key | cal_date      | traded
2 -----+-----+-----
3          1 | 2025-02-26    | true
4          1 | 2025-02-27    | true
5          1 | 2025-02-28    | true
6          1 | 2025-03-01    | false  <- closed for refurbishment
7          ... | ...           | false
8          1 | 2025-03-09    | false
9          1 | 2025-03-10    | true
10         1 | 2025-03-11    | true
```

6 Kestrel store 1 closed on 1 March 2025 and reopened on the 10th. Nine days, and the series simply stops.

THE TAKEAWAY

Real rows from the capstone database. 63 closures produce 1,266 missing store-days, so 412 stores have 475 trading spells between them.

Five rows, which is enough

```
1 d          traded
2 -----
3 2026-01-01 true
4 2026-01-02 true
5 2026-01-04 true      <- 3 January is missing
6 2026-01-05 true
7
8 -- expected: two islands, 1-2 Jan and 4-5 Jan
9 -- every technique in this module is testable here
```

8 Write this fixture first, get two islands, then point the query at seven million rows.

THE TAKEAWAY

Five rows and one gap. Add a duplicate row and the row-number difference fails; add a null and the value-change form fails. The whole module fits in a fixture.

The row-number difference. Subtract a counter from a date, and **runs share an answer.**

Solution one, in full

```
1 WITH numbered AS (  
2     SELECT store_key, cal_date,  
3           ROW_NUMBER() OVER (PARTITION BY store_key  
4                               ORDER BY cal_date) AS rn  
5     FROM store_day WHERE traded)  
6 SELECT store_key, MIN(cal_date) AS spell_from,  
7       MAX(cal_date) AS spell_to, COUNT(*) AS trading_days  
8     FROM numbered  
9  GROUP BY store_key, cal_date - INTERVAL (rn) DAY;
```

9 The whole technique is this line. Date minus counter is constant within a run.

5 Filter to the rows that exist first. The technique needs gaps, not flags.

THE TAKEAWAY

Nine lines, one of which is clever. It returns 475 spells on Kestrel's data in 0.03 seconds.

Why it works, on real rows

CAL_DATE	RN DATE MINUS RN
2025-02-26	1 2025-02-25
2025-02-27	2 2025-02-25
2025-02-28	3 2025-02-25
2025-03-10	4 2025-03-06
2025-03-11	5 2025-03-06
2025-03-12	6 2025-03-06
2025-03-13	7 2025-03-06

THE TAKEAWAY

Three rows share 25 February and four share 6 March. Neither anchor date is a real date in the data, and neither means anything. They are labels.

What the anchor actually is

- ◆ **It is the date the run would have started** if it had begun at row number one and run without a break.
- ◆ **Which is usually not a real date**, and is sometimes before the data begins. That is fine, and it confuses people.
- ◆ **You never output it.** It exists only to be grouped on, and MIN of the real date is what you report.
- ◆ **Any constant-within-a-run expression would do.** The date is convenient because subtraction is already defined on it.

THE TAKEAWAY

People try to interpret the anchor and get stuck. It is an arbitrary label that happens to look like a date, and looking like a date is an accident of the arithmetic.

Where the row-number difference breaks

- ◆ **Duplicate rows in the sequence.** Two rows on one date advance the counter twice, and the anchor shifts mid-run.
- ◆ **A non-integer sequence.** Timestamps, prices, anything where consecutive does not mean plus one.
- ◆ **Any tolerance rule.** A one-day gap that should not break a run cannot be expressed here at all.
- ◆ **Value-change runs,** where nothing is missing and the run breaks because a column changed.

THE TAKEAWAY

Four limits, and the first one is a data problem that looks like a query problem. The other three are cases where solution two is simply the right tool.

What a duplicate does to it

1	cal_date	rn	date minus rn
2	-----	---	-----
3	2026-01-01	1	2025-12-31
4	2026-01-02	2	2025-12-31
5	2026-01-02	3	2025-12-30 <- same date, counter moved
6	2026-01-03	4	2025-12-30

5 One duplicated day splits a three-day run into two islands, and every date in the data is still consecutive.

THE TAKEAWAY

The anchor shifts by exactly one day per duplicate and never shifts back. A run containing a duplicate is reported as two runs, with no gap anywhere in the dates.

A trading-spell report uses the row-number difference over Kestrel's store-day table. It returns 519 spells. Operations expect 475, because there are 412 trading stores and 63 refurbishment closures.

Every extra spell is adjacent to another spell for the same store, with no missing day between them. The store-day table was rebuilt last week from a new upstream feed.

What is the cause?

- ☐ A The rebuilt table has duplicate store-days, so ROW_NUMBER advances twice on one date and the anchor shifts.
- ☐ B The refurbishment table is missing 44 closures, so the expected figure of 475 is itself wrong.
- ☐ C The ORDER BY inside the window needs a tiebreaker, and without one the numbering is non-deterministic.
- ☐ D Use DENSE_RANK rather than ROW_NUMBER, which gives tied dates the same number.

◆ PAUSE AND DECIDE

What is the cause?

- A** Duplicate store-days
The counter advances and the date does not, so the anchor shifts back one day and never returns. One run becomes two, with no gap between them.
- B** Missing closures
Would produce spells separated by a real gap in the dates. The scenario says the extra spells are adjacent with no missing day.
- C** No tiebreaker
A real requirement, and here it would change which duplicate got which number rather than how many islands appear.
- D** Use DENSE_RANK
It does fix the island count, by papering over the duplication. The day counts stay wrong, because COUNT() still counts both rows.*

THE SIGNAL

"Adjacent, with no missing day between them." An island boundary where the dates are consecutive cannot be a gap. Something advanced the counter without advancing the date.

Flag the start of each run, then **count the flags so far**. That count is the group number.

Solution two, in full

```
1 WITH flagged AS (  
2   SELECT store_key, cal_date,  
3         CASE WHEN cal_date - LAG(cal_date) OVER w > 1  
4             OR LAG(cal_date) OVER w IS NULL  
5             THEN 1 ELSE 0 END AS starts_run  
6   FROM store_day WHERE traded  
7   WINDOW w AS (PARTITION BY store_key ORDER BY cal_date)),  
8 grouped AS (  
9   SELECT *, SUM(starts_run) OVER (  
10      PARTITION BY store_key ORDER BY cal_date  
11      ROWS UNBOUNDED PRECEDING) AS run_no  
12  FROM flagged)  
13 SELECT store_key, MIN(cal_date), MAX(cal_date), COUNT(*)  
14  FROM grouped GROUP BY store_key, run_no;
```

4 The first row of every partition has a null gap, so flag it explicitly. Module six.

THE TAKEAWAY

Longer, and every line says what it means. The rule that breaks a run is on line 3, in English, where a reviewer can disagree with it.

Why this one generalises

- ◆ **The rule is a predicate**, so it can be anything. More than one day, more than thirty minutes, a changed value, all three at once.
- ◆ **Tolerance is free**. Change the one to a three and a two-day gap no longer breaks a run.
- ◆ **Duplicates do not shift anything**, because the flag depends on values rather than on a counter.
- ◆ **And it works on any ordered type**. Timestamps, integers, sequence numbers, version numbers.

THE TAKEAWAY

Solution one is a trick that works on integers and dates. Solution two is a technique that works on anything, and the cost is four more lines.

Nothing is missing. The run breaks because
a value changed.

The two-row-number variant

```
1 SELECT member_durable_key, loyalty_tier, retail_month_key,  
2         ROW_NUMBER() OVER (PARTITION BY member_durable_key  
3                             ORDER BY retail_month_key)  
4         - ROW_NUMBER() OVER (PARTITION BY member_durable_key, loyalty_tier  
5                             ORDER BY retail_month_key) AS grp  
6 FROM snap_member_tier;
```

4 Two numberings, one overall and one within the value. Their difference is constant while the value does not change.

THE TAKEAWAY

A fourth technique, and a genuinely elegant one. It produces the same 413,636 spells and takes the same 0.4 seconds as the flag form.

Where the two-row-number form falls down

- ◆ **Nulls collect.** PARTITION BY the value puts every null month in one group, wherever those months actually sit.
- ◆ **So non-contiguous nulls produce wrong spells,** and that is scenario two's option C, which is why it fails.
- ◆ **It costs two sorts rather than one,** because the two window specifications differ. Module seven.
- ◆ **And it cannot take a tolerance either,** for the same reason the date version cannot. There is no predicate.

THE TAKEAWAY

Elegant, and narrower than it looks. Recognise it in old code, and reach for the flag form when writing something new.

Value-change islands

```
1 WITH marked AS (  
2   SELECT member_durable_key AS m, retail_month_key AS k, loyalty_tier AS v,  
3         CASE WHEN loyalty_tier IS DISTINCT FROM LAG(loyalty_tier) OVER w  
4             THEN 1 ELSE 0 END AS changed  
5   FROM snap_member_tier  
6   WINDOW w AS (PARTITION BY member_durable_key ORDER BY retail_month_key)),  
7 grouped AS (  
8   SELECT *, SUM(changed) OVER (PARTITION BY m ORDER BY k  
9       ROWS UNBOUNDED PRECEDING) AS spell_no FROM marked)  
10 SELECT m, spell_no, v, MIN(k), MAX(k), COUNT(*)  
11 FROM grouped GROUP BY m, spell_no, v;
```

3 IS DISTINCT FROM, not <>. The first row has a null previous value.

THE TAKEAWAY

Same shape as the trading spells. One predicate changed, and 1,050,000 rows become 413,636 spells.

The operator matters more here than anywhere

- ◆ **With not-equals, the first row of every partition is not flagged.** 50,000 members, 50,000 missing first spells.
- ◆ **And a transition into or out of a null is not flagged either,** so two genuinely different spells merge into one.
- ◆ **The merged spell then spans a value it never held,** which is a claim about the world rather than a rounding error.
- ◆ **IS DISTINCT FROM is two-valued.** It returns true against a null, which is what you meant both times.

THE TAKEAWAY

Module two's operator, doing the most consequential work it does anywhere in this course. A missing flag does not drop a row, it merges two spells.

A loyalty report builds tier spells from the monthly snapshot using LAG and a not-equals comparison. Tier is null for the months before a member joined the scheme, which is about 4% of the snapshot rows.

Finance find members whose Gold spell is recorded as running for eighteen months, where the scheme's own records show two separate Gold periods with a lapse in between. The spell count is short by roughly 39,000.

What is the cause?

- A Filter the null tiers out before building the spells, since a member with no tier has no spell to record.
- B Use the two-row-number method instead, which partitions by the value and therefore handles nulls correctly.
- C Not-equals returns unknown against a null, so a transition into or out of null is not flagged and two spells merge.
- D Coalesce the tier to 'None' before comparing, so that every month has a comparable value.

◆ PAUSE AND DECIDE

What is the cause?

A Filter the nulls out

Removes the rows and therefore removes the lapse, so the two Gold periods become adjacent and merge anyway. It makes the symptom worse.

C Three-valued logic

Gold, then null, then Gold. Neither transition is flagged, so one spell spans a lapse the member genuinely had.

B Two row numbers

PARTITION BY the tier collects all nulls into one group, wherever they sit. On non-contiguous nulls it produces worse spells, not better ones.

D Coalesce to 'None'

Works, and asserts a tier called None that the scheme does not have. It also hides the distinction between not joined and lapsed.

THE SIGNAL

"Two separate Gold periods with a lapse in between." Gold, null, Gold. Two unflagged transitions, one merged spell, and a member credited with status they did not hold.

Find the rows that **start** a run and the rows that **end** one, then pair them.

Solution three, in full

```
1 WITH starts AS (  
2   SELECT store_key, cal_date FROM store_day a WHERE traded  
3     AND NOT EXISTS (SELECT 1 FROM store_day b WHERE b.traded  
4       AND b.store_key=a.store_key AND b.cal_date = a.cal_date - 1)),  
5 ends AS (  
6   SELECT store_key, cal_date FROM store_day a WHERE traded  
7     AND NOT EXISTS (SELECT 1 FROM store_day b WHERE b.traded  
8       AND b.store_key=a.store_key AND b.cal_date = a.cal_date + 1))  
9 SELECT s.store_key, s.cal_date AS spell_from, MIN(e.cal_date) AS spell_to  
10  FROM starts s JOIN ends e  
11    ON e.store_key = s.store_key AND e.cal_date >= s.cal_date  
12 GROUP BY s.store_key, s.cal_date;
```

12 Each start pairs with the earliest end at or after it.

THE TAKEAWAY

No window functions at all. Runs on any engine ever built, and three times slower than solution one.

Which solution, when

ROW-NUMBER DIFFERENCE

Shortest

- One clever line
- Integers and dates only
- Breaks on duplicates
- No tolerance rule possible

FLAG AND RUNNING SUM

Most general

- The rule is a readable predicate
- Any ordered type
- Duplicates are harmless
- Tolerance costs one character

THE TIEBREAKER

Use the flag form by default. The row-number difference is worth recognising because it is everywhere in inherited code, and worth writing when the sequence is genuinely dense integers.

Module six's carry-forward was **this technique**, and it was not named.

COUNT as the group number

```
1 SELECT product_key, price_date, price,  
2         COUNT(price) OVER (PARTITION BY product_key  
3                             ORDER BY price_date  
4                             ROWS UNBOUNDED PRECEDING) AS grp  
5 FROM price_spine;  
6  
7 -- price_date  price  grp  
8 -- 2026-01-01   10    1    <- island 1 starts  
9 -- 2026-01-02  NULL    1  
10 -- 2026-01-03  NULL    1  
11 -- 2026-01-04  40     2    <- island 2 starts
```

2 COUNT of a column skips nulls, so it rises only on real values. That is the flag and the running sum in one function.

THE TAKEAWAY

No CASE expression and no LAG. COUNT(column) is already a conditional running total, because module two says it skips nulls.

Four flavours of run break

- ◆ **A missing row.** Trading days. Detected as a gap in a dense sequence, which is where the row-number difference works.
- ◆ **A gap in time.** App sessions. Detected with an interval predicate, and only the flag form can express it.
- ◆ **A changed value.** Tier spells. Detected with IS DISTINCT FROM, and nothing is missing anywhere.
- ◆ **A null.** Carry-forward. Detected by COUNT of the column, which is the shortest form of all four.

THE TAKEAWAY

Four break rules, four detection methods, one query shape after that. Naming which of the four you have is the first decision, and it takes five seconds.

Sessions, which is the next module

```
1 CASE WHEN event_ts - LAG(event_ts) OVER w > INTERVAL 30 MINUTE
2     OR LAG(event_ts) OVER w IS NULL
3     THEN 1 ELSE 0 END AS starts_session
4
5 -- 2,000,000 events, 50,000 members, no session column.
6 -- 1,985,019 sessions come out, which is very nearly
7 -- one event per session. Most app visits are one tap.
```

6 Measured from the capstone database. The number itself is the finding, and nobody expects it.

THE TAKEAWAY

Identical to the trading spells apart from the interval. The interesting part is not the query, it is that the answer tells you something about the app.

Why sessions are the hardest of the three

- ◆ **The rule is a business decision with no right answer.** Thirty minutes is a convention, not a fact about users.
- ◆ **Changing it changes every downstream number,** and the session count moves non-linearly with the threshold.
- ◆ **And the partition is a person, so there are fifty thousand of them.** Every one has a first event and an edge.
- ◆ **So the threshold belongs in one place,** named, where somebody can change it and see what moved.

THE TAKEAWAY

Trading spells have a fact underneath them: the store was shut. A session has only a convention, and the convention is usually undocumented.

Islands within islands

```
1 -- runs of consecutive trading days, per store AND per format
2 SUM(starts_run) OVER (PARTITION BY store_key, store_format
3                        ORDER BY cal_date
4                        ROWS UNBOUNDED PRECEDING)
5
6 -- and the flag itself must partition the same way
7 CASE WHEN cal_date - LAG(cal_date) OVER (
8       PARTITION BY store_key, store_format ORDER BY cal_date) > 1
9       THEN 1 ELSE 0 END
```

- 8 Both windows must use the same partition. A flag computed on one partition and summed on another produces nonsense silently.

THE TAKEAWAY

The most common bug in a multi-key islands query. Use the named WINDOW clause and the two cannot drift apart.

Partitioning by something that changes

- ◆ If the **partition key** can change mid-run, it is not a partition key. It is a value, and a change in it breaks the run.
- ◆ A **store that changes region mid-year** is the classic case. Partition by region and one spell becomes two, silently.
- ◆ **Decide which you meant.** Runs of trading days per store, or runs of trading days per store within one region?
- ◆ **Both are defensible**, and they give different answers, and nothing in the query says which was intended.

THE TAKEAWAY

Kestrel restructured to seven regions in April. Any spell query partitioned by region splits every store's history at that date, and correctly so.

Everything so far found the islands.
Sometimes you want the gaps.

Gaps, from the islands you already have

```
1 WITH spells AS ( -- the islands, from any of the three methods
2   SELECT store_key, spell_from, spell_to FROM trading_spells
3 )
4 SELECT store_key,
5        spell_to + 1                                AS gap_from,
6        LEAD(spell_from) OVER w - 1                 AS gap_to,
7        LEAD(spell_from) OVER w - spell_to - 1      AS gap_days
8   FROM spells
9  WINDOW w AS (PARTITION BY store_key ORDER BY spell_from)
10 QUALIFY LEAD(spell_from) OVER w IS NOT NULL;
```

- 6 The gap runs from the day after one island ends to the day before the next begins. LEAD does it in one line.
- 10 The last island of every store has no following island, so it produces no gap. QUALIFY removes those rows.

THE TAKEAWAY

Islands first, gaps second. Computing the gaps directly is possible and always harder, because a gap is defined by the things on either side of it.

The gaps you cannot see

- ◆ **A gap before the first island does not exist in the data.** Nothing precedes it, so nothing detects it.
- ◆ **Nor does a gap after the last one.** A store closed permanently in June looks like a store whose data simply ends.
- ◆ **Anchor to a known window**, so the period start and end act as a virtual island on either side.
- ◆ **Which is the same fix as module six's series completion**, applied to the edges rather than to the middle.

THE TAKEAWAY

This is why gaps are computed against a declared reporting period rather than against the data. Absence at the edges is invisible unless you supply the edges.

A minimum island length, written out

```
1 SELECT store_key, MIN(cal_date) AS spell_from,  
2         MAX(cal_date) AS spell_to, COUNT(*) AS trading_days  
3 FROM grouped  
4 GROUP BY store_key, run_no  
5 HAVING COUNT(*) >= 7;           -- at least a week
```

5 HAVING, so the filter applies to the island. A WHERE here would filter rows and change which islands exist.

THE TAKEAWAY

Module one, at the level of a technique. Filtering rows before grouping can split one island into two, or merge two into one, and the count you asked about changes.

Anchoring to the reporting window

```
1 WITH bounded AS (  
2   SELECT store_key, cal_date FROM store_day WHERE traded  
3   UNION ALL  
4   SELECT store_key, DATE '2025-02-02' FROM dim_store -- day before  
5   WHERE is_trading  
6   UNION ALL  
7   SELECT store_key, DATE '2026-09-01' FROM dim_store -- day after  
8   WHERE is_trading)  
9 -- now a gap at either edge has something on both sides of it
```

7 Two virtual trading days, one either side of the window, so an edge gap becomes an ordinary gap.

THE TAKEAWAY

A store closed from the first day of the period has no gap in the data, because nothing precedes it. Supplying the edges is what makes the absence visible.

Real rules are rarely 'consecutive'. They are
'close enough'.

Three tolerances, one query shape

```
1 -- consecutive days
2 CASE WHEN cal_date - LAG(cal_date) OVER w > 1 THEN 1 ELSE 0 END
3
4 -- a gap of up to a week does not break the run
5 CASE WHEN cal_date - LAG(cal_date) OVER w > 7 THEN 1 ELSE 0 END
6
7 -- thirty minutes of silence ends a session
8 CASE WHEN event_ts - LAG(event_ts) OVER w
9         > INTERVAL 30 MINUTE THEN 1 ELSE 0 END
10
11 -- the value changed
12 CASE WHEN tier IS DISTINCT FROM LAG(tier) OVER w THEN 1 ELSE 0 END
```

12 Four completely different business rules. The rest of the query is identical in all four cases.

THE TAKEAWAY

Trading spells, activity spells, app sessions and tier spells. One query shape, four predicates, and the predicate is the only thing anybody needs to review.

Merchandising want store trading spells where a closure of two days or fewer does not break the spell, because short closures are stocktakes rather than refurbishments.

The existing report uses the row-number difference. An analyst has spent a morning trying to express the two-day tolerance in it and asks whether it is possible.

What should they be told?

- A Divide the date by three before subtracting the row number, which absorbs gaps of up to two days.
- B The tolerance is a predicate, and only the flag-and-running-sum form has one. Change the 1 to a 2.
- C Use the start-and-end pairing form, and change the anti-join to look back two days instead of one.
- D Pre-fill the missing days for gaps of two or fewer, then the row-number difference works unchanged.

◆ PAUSE AND DECIDE

What should they be told?

A Divide the date

Produces bands rather than runs. Two dates three days apart can land in one band and two dates one day apart in different ones.

C Widen the anti-join

Closer than it looks, and it breaks on a three-day gap that contains a traded day. The pairing then joins across a real closure.

B Change the predicate

The rule lives in one place, as a number a stakeholder can read and argue with. Tolerance costs one character.

D Pre-fill short gaps

Works, and changes the data to fit the technique. The day counts now include days the store was shut, which is a different measure.

THE SIGNAL

"Spent a morning trying to express it." The row-number difference has no place to put a rule, because its group number comes from arithmetic rather than from a predicate.

Islands over a numeric sequence

```
1 -- missing invoice numbers, missing order IDs, missing versions
2 WITH n AS (
3     SELECT invoice_no,
4            invoice_no - ROW_NUMBER() OVER (ORDER BY invoice_no) AS grp
5     FROM fact_invoice)
6 SELECT MIN(invoice_no) AS run_from, MAX(invoice_no) AS run_to,
7        COUNT(*) AS run_length
8 FROM n GROUP BY grp;
```

4 Integers subtract from integers. No interval arithmetic, and the same technique exactly.

THE TAKEAWAY

The row-number difference is at its best here. A dense integer sequence is precisely what it was designed for, and there is no tolerance rule to want.

Find the gap, or fill it

DETECT

Report the gaps

- The absence is the finding
- Closed days, missing invoices
- Output is one row per gap
- Nothing is invented

COMPLETE

Fill the series

- The absence is an obstacle
- Moving averages, LAG by day
- Output is one row per period
- Zero or null is asserted

THE TIEBREAKER

Module six wanted the series completed so LAG would mean 'yesterday'. This module often wants the gaps kept, because the gap is the answer.

Islands with a minimum length

- ◆ **Filter after grouping, not before.** A HAVING clause on the island's length, once the island exists.
- ◆ **Filtering rows first changes the islands,** because removing a row can split one run into two or merge two into one.
- ◆ **Which is module one, at the level of a technique.** The filter's position decides what is being counted.
- ◆ **And short islands are often the interesting ones,** so check whether you want them removed or reported.

THE TAKEAWAY

A three-day spell is either noise to be filtered or a stocktake to be investigated. The query cannot tell, and whichever you choose should be written down.

Three assertions for any islands query

- ◆ **Coverage.** The sum of island lengths equals the number of rows that went in. Nothing lost, nothing counted twice.
- ◆ **No overlap.** No two islands for the same key share a date. A grouping key that is not constant within a run fails this.
- ◆ **No adjacency.** No two consecutive islands are separated by less than the gap rule. Two islands that should be one fail this.
- ◆ **Together they are complete.** A result passing all three is correct, and any one of them alone is not enough.

THE TAKEAWAY

The capstone harness checks exactly these three on d03, d04 and d05. They catch every failure in this module, including the duplicate one that produced 519 spells.

The longest island per group

```
1 WITH spells AS ( ... the islands, however you built them ... )
2 SELECT store_key, spell_from, spell_to, trading_days
3     FROM spells
4     QUALIFY ROW_NUMBER() OVER (
5         PARTITION BY store_key
6         ORDER BY trading_days DESC, spell_from) = 1;
```

6 A tiebreaker, because two spells can be the same length. Module two, on the output of this module.

THE TAKEAWAY

The most common follow-up question, and it is module four applied to the islands rather than to the rows. Nothing new, and the tiebreaker is still compulsory.

Other things people ask next

- ◆ **The current island.** The one whose end is the last day in the window. Filter, do not assume it is the longest.
- ◆ **Islands overlapping a date range.** A standard interval overlap test, which is module fifteen.
- ◆ **Total days in islands versus total days in the window.** A coverage percentage, and one line.
- ◆ **How many times did this break?** Islands minus one, per group, which is the gap count.

THE TAKEAWAY

All four are one line on top of the islands. Building the islands as their own named block is what makes all four cheap rather than four separate queries.

The three, as one query

```
1 SELECT
2   (SELECT SUM(trading_days) FROM spells)
3   = (SELECT COUNT(*) FROM store_day WHERE traded)      AS coverage_holds,
4   NOT EXISTS (SELECT 1 FROM spells a JOIN spells b
5               ON a.store_key = b.store_key
6               AND a.spell_from < b.spell_from
7               AND a.spell_to >= b.spell_from)           AS no_overlap,
8   NOT EXISTS (SELECT 1 FROM (
9     SELECT store_key, spell_from,
10            LAG(spell_to) OVER (PARTITION BY store_key
11                               ORDER BY spell_from) AS prev_to
12   FROM spells)
13   WHERE spell_from - prev_to <= 1)                     AS no_adjacency;
```

- 13 Two islands one day apart should have been one island. This is the check nobody writes, and it catches the merge failures.

THE TAKEAWAY

Three booleans, one query, run after every islands query you write. On Kestrel's 475 spells all three are true, and on the duplicated feed the first one is false.

What an islands query costs

- ◆ **One sort of each partition**, for the window forms. Everything else is a scan and a grouped aggregate.
- ◆ **The start-and-end form does a lookup per row**, twice, and then a join. Three times slower here, and worse as data grows.
- ◆ **Reduce to the grain of the question first**. Trading spells are about days, so aggregate to days before looking for runs.
- ◆ **That last point dominates everything else**. 7 million sales lines become 237 thousand store-days.

THE TAKEAWAY

Measured on the capstone database: 0.03 seconds for the row-number difference, 0.09 for the flag form, 0.10 for start-and-end pairing. The grain decision is worth more than all of that.

A trading-spell query runs the row-number difference directly over `fact_sales_line`, partitioning by store and ordering by `sold_ts`, with a `DISTINCT` to reduce to one row per store per day afterwards.

It takes several minutes and spills. The store-day table, holding one row per store per day, already exists and is refreshed nightly. The answer the query produces is correct.

What is the right change?

- ☐ A Add an index on `sold_ts`, so that the sort inside each partition is cheaper.
- ☐ B Switch to the start-and-end pairing form, which needs no sort of the partition.
- ☐ C Increase the cluster, since seven million rows sorted by store is inherently memory-bound.
- ☐ D Run it over the store-day table. The question is about days, so reduce to days before looking for runs.

◆ PAUSE AND DECIDE

What is the right change?

A Index sold_ts

Helps the sort and still sorts seven million rows to produce four hundred and seventy five answers. The index is not the ratio that matters.

C Bigger cluster

Pays to sort rows that should never have entered the query. The table grows by about four thousand rows a day.

B Start-and-end pairing

It does avoid the partition sort and replaces it with two correlated lookups per row. On seven million rows that is worse, not better.

D Use the store-day table

Thirty times fewer rows, the grain the question is actually asked at, and the table is already built and refreshed nightly.

THE SIGNAL

"A DISTINCT to reduce to one row per store per day afterwards." The query already knows it wants day grain. It is simply doing the reduction at the end rather than the start.

Spells are usually **a modelling output**, not a query. They belong in a table.

When to store the spells

- ◆ **When more than one report needs them.** The second consumer is the moment, and it always arrives.
- ◆ **When the source is large and the spells are small.** 237,000 store-days become 475 spells, which is a 500-fold reduction.
- ◆ **When the rule is contested.** A stored table has one definition, and an argument about it happens once.
- ◆ **And not when the rule varies by question,** because then you are storing one answer to a question people ask differently.

THE TAKEAWAY

Kestrel's 475 trading spells fit in a table you could print. Rebuilding them in six dashboards is six chances for the tolerance rule to be different in one of them.

The spells table

```
1 CREATE OR REPLACE TABLE dim_store_trading_spell AS
2 SELECT store_key,
3        MIN(cal_date) AS spell_from,
4        MAX(cal_date) AS spell_to,
5        COUNT(*)      AS trading_days,
6        ROW_NUMBER() OVER (PARTITION BY store_key
7                          ORDER BY MIN(cal_date)) AS spell_no
8 FROM grouped
9 GROUP BY store_key, run_no;
```

7 A window over a grouped result, from module one. The spell number is only meaningful once the spells exist.

THE TAKEAWAY

475 rows, with a stated grain of one row per store per spell. That table is now a dimension, and everything downstream joins to it on a date range.

Every technique here invents **a column that rows in one run happen to share**. That is all any of them do.

Recognising it in inherited code

- ◆ **A date minus a ROW_NUMBER, in a GROUP BY.** That is solution one, and it will not be commented.
- ◆ **A SUM of a CASE returning 1 or 0, over an ordered frame.** That is solution two, and neither will it.
- ◆ **Two ROW_NUMBERS subtracted.** That is the value-change variant, and it looks like a mistake.
- ◆ **All three mean the same thing.** Somebody was looking for runs, and there is no keyword that would have said so.

THE TAKEAWAY

This is the most common unrecognised pattern in analytics SQL. Being able to name it on sight is worth as much as being able to write it.

The fixture, with the failure cases in it

```
1 -- five rows, one gap                two islands
2 (1,'2026-01-01'),(1,'2026-01-02'),(1,'2026-01-04'),(1,'2026-01-05')
3
4 -- add a duplicate                    row-number difference now gives 3
5 (1,'2026-01-02')
6
7 -- add a second group                catches a missing PARTITION BY
8 (2,'2026-01-01'),(2,'2026-01-02')
9
10 -- add a null value                  catches <> instead of IS DISTINCT FROM
11 (2,'2026-01-03',NULL)
```

7 Two groups, always. One group cannot catch a missing PARTITION BY, and that is the commonest defect.

THE TAKEAWAY

Eight rows, four failure modes, and every one of them visible by eye. Production data will not test any of these, because it either has them or it does not.

Testing it properly

- ◆ **Two groups in the fixture, always.** One group cannot catch a missing PARTITION BY, and that defect is invisible otherwise.
- ◆ **A duplicate row,** because it separates the row-number difference from the flag form immediately.
- ◆ **A run of length one,** because a single-row island is an edge case and people forget it exists.
- ◆ **And a gap at the very start,** which tests whether the first row of each partition was flagged.

THE TAKEAWAY

Four rows of thought, and they cover every scenario in this module. The capstone's three islands deliverables fail on exactly these four things.

What Part 4 does with this

- ◆ **Module fourteen sessionises events**, which is this technique with an interval predicate and two million rows.
- ◆ **Module fifteen joins on the spells**, because a spell is an interval and intervals overlap, merge and nest.
- ◆ **Both assume you can produce the spells**. This is the module they stand on.
- ◆ **And three of the capstone's ten deliverables are this**, which is why it is the longest module in the part.

THE TAKEAWAY

Part 4 is one technique and two applications of it. If this module is solid, the other two are mostly about what the runs then mean.

The three things that go wrong most

- ◆ **Wrong grain.** Looking for runs of days over a table whose rows are sales lines. Reduce first, always.
- ◆ **Missing partition.** One store's last day joining to the next store's first. Two groups in the fixture catches it.
- ◆ **Unflagged first row.** Every partition's opening run never starts, so the first spell of every group disappears.
- ◆ **All three are caught by the coverage assertion**, which is one line and which almost nobody writes.

THE TAKEAWAY

Three defects, one assertion. The sum of island lengths against the input row count catches every one of them, and it costs a single query.

The three solutions, summarised

	ROW-NUMBER FLAG+SUM START/END
Group number from	arithmetic a predicate no group number
Works on	dates, ints anything anything
Tolerance rule	no yes awkward
Value-change runs	variant only yes awkward
Survives duplicates	no yes yes
On Kestrel's 237k rows	0.03s 0.09s 0.10s

THE TAKEAWAY

The flag form loses on one row of this table and wins on four. Three hundredths of a second is not a reason to choose a technique that cannot express your rule.

Writing one, in four steps

01

Grain

What does one row mean? Reduce to the grain the question is asked at, first.

02

Partition

Runs of what, within what? Store, member, product. This is the PARTITION BY.

03

Rule

State what breaks a run in one English sentence. That sentence becomes the predicate.

04

Assert

Coverage, no overlap, no adjacency. Three booleans, and all three must hold.

THE TAKEAWAY

Step three is the one to do out loud and before writing anything. If the sentence is hard to say, the query will be hard to write and harder to review.

The confusions, settled

SYMPTOM	THE CAUSE
More islands than expected, no gap between them	Duplicate rows advancing the row-number counter.
Two spells merged into one	Not-equals against a null, so the transition was not flagged.
The first spell of every group is missing	The first row has a null previous value and was not flagged.
Islands overlap each other	The grouping key is not constant within a run.
The anchor date looks wrong	It is a label, not a date. You are not meant to read it.
Tolerance cannot be expressed	You are using the row-number difference. It has no predicate.

THE TAKEAWAY

Rows two and three are the same defect at two different places, and both come from module two rather than from this module.

Five habits

- ◆ **Reduce to the question's grain before looking for runs.** It is worth more than every other optimisation combined.
- ◆ **Default to the flag-and-running-sum form.** The rule ends up on one line where somebody can disagree with it.
- ◆ **Use IS DISTINCT FROM for every value comparison,** because the previous value is nullable by construction.
- ◆ **Flag the first row of every partition explicitly.** It has no predecessor and it starts a run.
- ◆ **Assert coverage, overlap and adjacency,** every time. Three booleans, and they are complete.

THE TAKEAWAY

The third and fourth habits are both module two. This module's own contribution is the group number, and that part is the easy half.

You have the technique. **Module fourteen** applies it to events, where the run is measured in minutes.